

# SHARP: An Energy-Efficient Spike-Based 3-D Recognition Processor With Hierarchical Hybrid Sparsity Awareness on Point Cloud

Ziyang Shen<sup>1b</sup>, *Graduate Student Member, IEEE*, Zhipeng Liao<sup>1b</sup>, *Student Member, IEEE*,  
Sitan Shen<sup>1b</sup>, *Student Member, IEEE*, Chaoming Fang<sup>1b</sup>, *Student Member, IEEE*,  
Fengshi Tian<sup>1b</sup>, *Graduate Student Member, IEEE*, Jie Yang<sup>1b</sup>, *Senior Member, IEEE*,  
and Mohamad Sawan<sup>2b</sup>, *Life Fellow, IEEE*

**Abstract**—LiDAR-based 3D recognition leverages exceptional spatial granularity and broad sensory coverage to achieve high-precision perception in complex outdoor environments. Despite these advantages, the prohibitive computational workload makes real-time deployment on resource-limited edge devices a significant challenge. High-fidelity point cloud processing typically demands deep network structure, thus requiring massive operations and consuming many resource-intensive multipliers in conventional DNN accelerators. Moreover, conventional data representations and computing architectures become suboptimal under the dynamic sparsity among partitioned data tiles. In this paper, we propose SHARP, a novel spiking reconfigurable accelerator featuring hierarchical hybrid sparsity awareness on 3D point cloud based recognition tasks. A hierarchical hybrid sparsity codec is introduced to minimize data volume, significantly reducing external memory access. Moreover, a reconfigurable rulebook-GEMM fused spiking engine with flexible scheduling is developed to bridge the gap between fine-grained rulebook-based indexing and parallel execution, maintaining high performance across diverse sparsity distributions. The spike-based engine also secures low power consumption through discrete 1-bit data transmission and multiplier-free micro-architectural design. Fabricated in 28nm CMOS technology, the prototype chip achieves a peak system-level energy efficiency of 83.2 TOPS/W. Evaluation results show that our approach outperforms state-of-the-art 3D point cloud processors with a 2.16x boost in energy efficiency and the least energy consumption per frame, making it highly suitable for edge intelligence.

**Index Terms**—Point cloud, 3-D recognition acceleration, spiking neural network, reconfigurable accelerator, dynamic sparse processing.

Received 26 April 2026; revised 15 June 2026 and 20 July 2026; accepted 31 July 2026. This work was supported by the Brain Science and Brain-Like Intelligence Technology—National Science and Technology Major Project under Grant 2022ZD0208805, in part by the “Pioneer” and “Leading Goose” Research and Development Program of Zhejiang under Grant 2024C03002, in part by the National Natural Science Foundation of China under Grant W2431058, in part by the Westlake University Laboratory Talent Acquisition Special Funding under Grant 103186021801, and in part by Laboratory National Foreign Expert Project under Grant 10318A031901. This article was recommended by Associate Editor J. Tian. (Ziyang Shen and Zhipeng Liao are co-first authors.) (Corresponding authors: Jie Yang; Mohamad Sawan.)

Ziyang Shen, Zhipeng Liao, Sitan Shen, Chaoming Fang, Jie Yang, and Mohamad Sawan are with the Center of Excellence in Biomedical Research on Advanced Integrated-on-chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, Hangzhou 310030, China (e-mail: yangjie@westlake.edu.cn; sawan@westlake.edu.cn).

Fengshi Tian is with The Hong Kong University of Science and Technology, Hong Kong, SAR.

Digital Object Identifier 10.1109/TCSI.2026.3720571

## I. INTRODUCTION

POINT cloud-based 3D recognition extracts semantic features from sparse, unstructured spatial coordinates. It maps raw geometric data into rich environmental representations, offering a robust solution for 3D perception. Using point clouds as a unified input, 3D recognition encompasses three primary tasks: classification, detection, and segmentation. As illustrated in Fig. 1, specific outputs are generated for different tasks, including category vectors to classify objects, bounding boxes to locate objects, and labeled point sets to segment the scene.

Due to their high dimensionality and expansive sensing range, point clouds inherently capture rich spatial details. While these features are crucial for enhancing perception capabilities in complex environments, they also impose immense computational demands. For real-world edge applications, such as autonomous driving [1], [2], [3] and embodied intelligence [4], this heavy computational burden poses a huge challenge to the computing efficiency on strictly power- and resource-constrained platforms. Previous efforts have been made to alleviate such bottleneck [5], [6], [7], [8], [9], [10], [11]. However, to fully achieve real-time and high-precision inference on resource-constrained edge, three fundamental challenges still need to be addressed, as shown in Fig. 1:

The first challenge is the inefficient computation primitive in DNN-based 3D point cloud processing. Because 3D sensory data spans an ultra wide physical range with vast empty spaces, capturing meaningful global context in 3D requires hierarchically deep network structures and large convolutional kernels. This introduces enormous operations and necessitates massive multiply-accumulate (MAC) arrays on the underlying hardware. However, in conventional deep neural network (DNN) accelerators, the energy efficiency of MAC units is bottlenecked by their multipliers, which typically account for approximately 87% of the power consumption and 89% of the silicon area within each unit [12]. These resource-intensive logic components, when organized into dense arrays, result in excessive energy overhead that is fundamentally incompatible with the stringent power constraints of edge-side deployment. Consequently, a fundamental paradigm shift in architecture is required to circumvent such inherent hardware limitation.

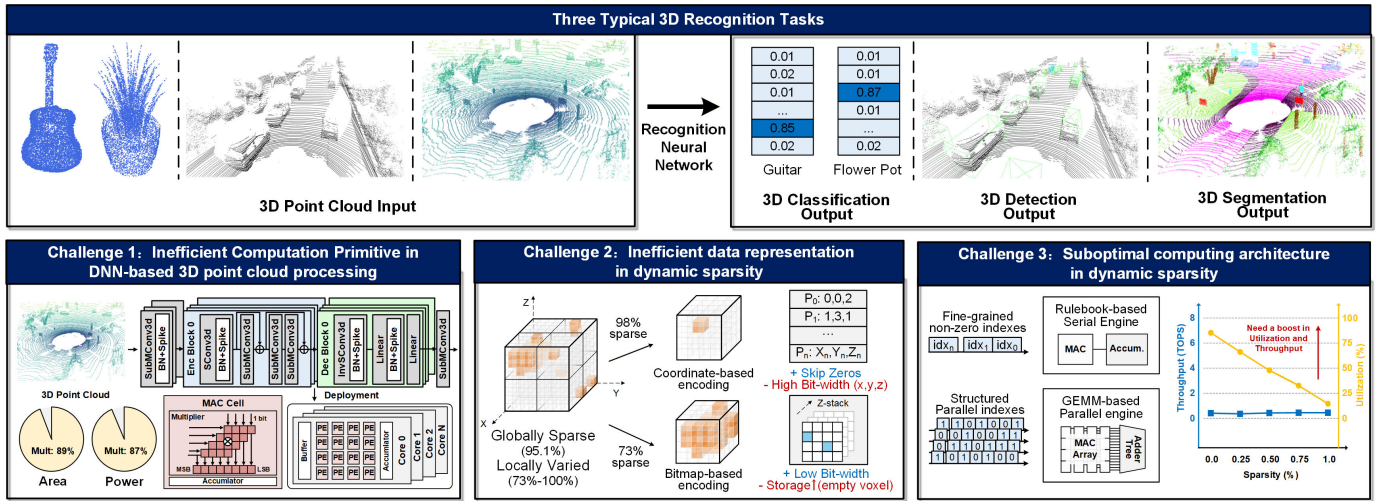


Fig. 1. Three main challenges faced when deploying 3D point cloud recognition algorithms onto specialized hardware.

The second challenge stems from inefficient point cloud data representation in dynamic sparsity. To manage limited on-chip memory, point clouds are typically partitioned into smaller tiles. This dynamic sparsity manifests in two ways: first, feature sparsity varies significantly across different layer types; second, even when the global sparsity of certain layers is extremely high, the local sparsity within each partitioned tile fluctuates significantly [8]. Such variations pose a major challenge to intermediate data representation. Typically, intermediate data is represented in either sequential bitmaps or disordered coordinate formats. In the bitmap format, while low bit-width data enables a compact representation [7], the high dimensionality and fine spatial granularity of 3D point clouds—characterized by vast empty backgrounds—can cause the data volume to expand uncontrollably. Conversely, storing data as multi-bit coordinates effectively skips zero elements [13], yet it risks a data scale explosion as the density of non-zero points increases. Therefore, a novel data representation that remains efficient across all sparsity levels is urgently required.

The third challenge is the suboptimal computing architecture in dynamic sparsity. The fluctuating sparsity level not only causes data representation issue, but also creates a fundamental conflict in computing units between exploiting data sparsity and maintaining computational parallelism. Specifically, while mechanisms such as rulebook-based indexing [14] facilitate fine-grained sparse processing, they often perform poorly under low sparsity level because of the low efficiency of gathering required data for computation and long sequential addition latency [6]. Conversely, GEMM-based computation offers faster execution of dense data, but results in redundant calculations of structured 3D point clouds with billions of empty voxels composed under high sparsity level. This necessitates a delicate balance between these two computing paradigms for a higher utilization and throughput under all sparsity level.

To tackle the challenges above, we propose a 28nm spiking 3D recognition accelerator called SHARP. Three main features are proposed in this work:

- 1) **Spike-based Algorithm-Hardware Co-optimization**  
Inspired by neuro-biology, spike-based computing has been proved to be a promising power-efficient paradigm [15], [16], [17], [18], [19]. In SHARP, we implemented an algorithm-hardware co-optimized spiking neural network acceleration system, which facilitates massive information transmission via asynchronous, discrete 1-bit spikes and substitutes MAC operations with highly energy-efficient additions.
- 2) **Hierarchical Hybrid Sparsity Codec (HHSC)**: A set of hybrid sparsity encoder and decoder with a hierarchical decoupled global-local coordinate system is proposed to minimize sparse data volume through efficient spatial partitioning. By abstracting joint global information and adaptively switching between coordinate and bitmap formats based on tile density, the HHSC significantly compresses indexing bit-widths, effectively reducing the on-chip SRAM footprint and power-hungry External Memory Access (EMA) by 65.3% on average.
- 3) **Reconfigurable Spiking Engine (RSE)**: To balance fine-grained sparsity with parallelism, a dual-mode reconfigurable engine is developed featuring rulebook-based sparse logic with independent serial adders and GEMM-based (General Matrix Multiplication) parallel execution with adder trees, maximizing hardware utilization and throughput across diverse sparsity distributions. Furthermore, a hybrid output scheduler is introduced to seamlessly fuse the disparate data formats and orchestrate the output dataflow for efficient accumulation. By bridging the gap between irregular sparse indexing and parallel execution, the proposed engine architecture improves hardware utilization by 81.2% and reduces overall computation latency by 81.3%.

## II. PRELIMINARIES AND BACKGROUND

### A. Spiking Point Cloud Based 3D Recognition Workflow

Spiking neural network (SNN), one kind of brain-inspired neuromorphic computing, has emerged as a novel and

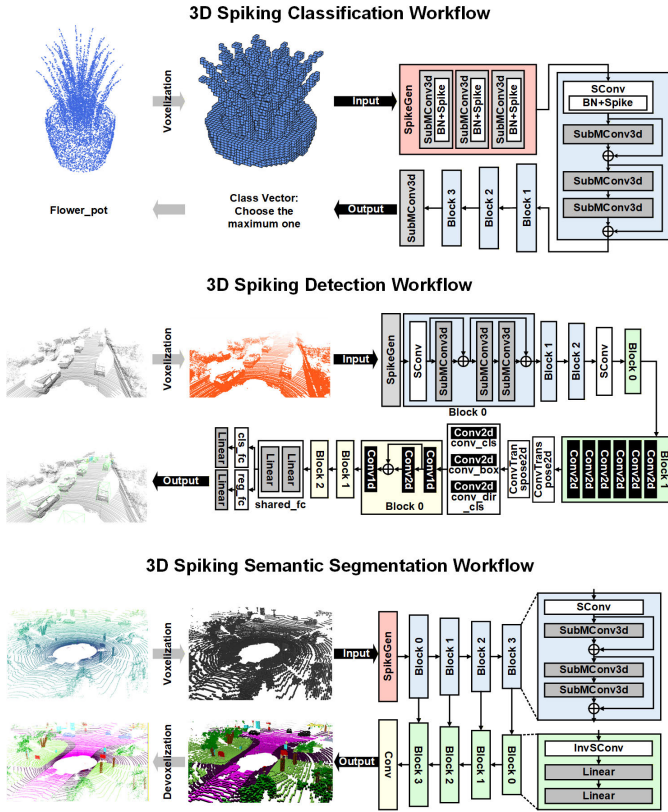


Fig. 2. Workflow of spiking point cloud based 3D recognition task.

promising paradigm. It utilizes discrete binary spikes to represent intermediate feature maps and mimics neural dynamics in human brain to update artificial neuron states, making it inherently suitable for ultra-low power edge deployment. A properly trained SNN has been proved to be able to achieve comparable performance with DNN on point cloud based 3D recognition tasks [20], [21], [22], [23], [24], [25], which shows its practical capability. The fundamental computation of SNN resides in the Leaky Integrate-and-Fire (LIF) neuron model, which governs the accumulation and firing of information. The discrete-time membrane potential  $u_i[t]$  of a neuron  $i$  is updated according to the following recurrence relation:

$$u_i[t] = \lambda \cdot u_i[t-1] + \sum_j w_{ij} s_j[t] \quad (1)$$

$$s_i[t] = \epsilon(u_i[t] - V_{th}) \quad (2)$$

where  $\lambda$  represents the decay constant,  $w_{ij}$  denotes the synaptic weights,  $s_j[t] \in \{0, 1\}$  signifies the input spike activity from the  $j$ -th pre-synaptic neuron,  $\epsilon(x)$  denotes the spike firing function, and  $V_{th}$  is the firing threshold.

Fig. 2 illustrates the algorithmic workflow for spiking point cloud based 3D recognition [25] adopted in this work. The workflow encompasses three representative recognition tasks: classification, detection, and semantic segmentation, which is unified under an end-to-end spike-based processing paradigm. All point cloud input are first voxelized and converted into spikes. For 3D object classification, 3D submanifold and sparse convolutional layers are cascaded to extract hierarchical geometric features into a 40-element category vector with

one-hot code to show the predicted category. For 3D object detection, a 3D sparse convolutional backbone is utilized for initial feature extraction, followed by a projection onto the 2D plane where 2D convolutional layers refine deep and multi-scale features to output precise bounding box coordinates and localization parameters. As to 3D semantic segmentation, an asymmetric encoder-decoder structure similar to U-Net is employed, with the encoder hierarchically capturing multi-scale spatial features and the decoder restoring spatial resolution through skip-connection-based fusion to generate a point-level labeled point cloud that partitions the 3D environment into distinct semantic regions.

### B. Point Cloud Processing Paradigms

Point cloud representation paradigms generally fall into two categories: point-based and voxel-based approaches. Point-based methods operate directly on raw coordinates, preserving high spatial granularity and capturing fine-grained geometric features [5], [9], [10], [11], [26], [27], [28]. However, for large-scale outdoor scenarios, the computational and memory overhead of these methods often becomes prohibitive. Furthermore, the unstructured and scattered nature of raw points hinders the direct application of standard 3D convolution kernels. In contrast, voxel-based methods mitigate these challenges by partitioning the 3D space into structured grids, where features from proximal points are aggregated into discrete voxels [29], [30], [31], [32]. Due to reduced spatial complexity, this paradigm is more amenable to resource-constrained edge deployment and is therefore adopted in this work.

To efficiently process voxel-based point cloud data, two predominant computing paradigms are widely adopted: GEMM-based and rulebook-based computation. In the GEMM-based paradigm, the accelerator typically scans the entire receptive field and applies pre-trained weight kernels to adder tree arrays for GEMM. This approach can further seek to explore sparsity by employing zero-skipping logic to bypass redundant computations and memory accesses.

Conversely, rulebook-based computation processes individual non-zero input elements and establishes dynamic coordinate mappings—often referred to as rulebooks—between input, weight, and output indices based on convolution kernels. The resulting partial sums are subsequently accumulated in serial adders to yield the final output membrane potential.

### C. Related Works

Existing voxel-based 3D point cloud processors have achieved promising levels of performance and real-time capabilities. [6], [7], [8], [13], [33]. Cao and Gu [6] presented a hardware-friendly hopping-index rulebook mechanism with an octree-based data structure to constrain the search range. Similar in data management, Lyu et al. [33] adopted octree-encoding for efficient map searches, and proposed a specialized sparsity-aware computing core with gather units to prefetch valid data and maximize throughput. To address the challenges of multi-operator support, Feng et al. [7] focused on boosting GEMM-based neighbor search efficiency

There also exists neuromorphic accelerators working on point cloud applications [34], [35]. Lee et al. [34] proposed a point-based neuromorphic point cloud processor with an event-driven bit-serial computing unit and an output pre-cognition unit, exploring timestep skipping and reduction. Wang et al. [35] also worked on point-based approaches and proposed an FPGA-based neuromorphic point cloud processor with tailored point-based optimization unit and quantization exploration.

This work presents a spiking vision accelerator for 3D recognition, as shown in Fig. 3. The accelerator consists of a RISC-V top controller to orchestrate the dataflow, DMA controller for direct data transmission between on/off-chip memory, and 24 sparse cores for data processing. Each sparse core shows two explicit dataflow paths processing coordinates and bitmaps respectively, and four features:

- 1) A hierarchical hybrid sparsity encoder is designed to minimize coordinate volume by partitioning the global voxel space into uniform tiles. To further optimize coding efficiency across fluctuating sparsity levels, the proposed encoder adaptively selects between coordinate and bitmap formats based on the count of active voxels within each tile. By dynamically employing the most bit-efficient representation for varying densities, the proposed encoder significantly mitigates power-hungry EMA.
- 2) A rulebook-GEMM fused spiking engine is designed to execute hybrid sparse computations through specialized reconfigurable compute cores. The core can be configured into a pure adder tree array for fast parallel execution or the combination of adder trees and serial adders for flexible sparsity exploration under different conditions. This dual-mode reconfigurable architecture bridges the gap between fine-grained sparse indexing and coarse-grained parallel processing, ensuring optimal utilization and execution time across diverse spatial distributions.
- 3) To manage the dual output streams, a hybrid output scheduler is implemented to process both coordinate-format results from the rulebook-based serial adders and bitmap-format results from the GEMM-based adder trees. Also, output boundary merging is considered with a reconfigurable structure. This feature enables efficient online merging within a constrained on-chip resource budget, streamlining the transition from disparate computations to integrated feature maps.
- 4) A Hierarchical hybrid sparsity decoder is proposed to generate spikes from merged partial sums and compress

The diagram illustrates the system architecture of the proposed RISC-V SoC, divided into four main functional blocks and supporting components.

- RISC-V CPU:** Contains a Processor, Scaler Unit, and Core Config. Buffer.
- DMA Controller:** Manages data flow between the CPU and the processing blocks, including Input Coord. DMA, Input Spike Feature DMA, Weight DMA, Output Coord. DMA, and Output Spike Feature DMA.
- Feature 1: Hierarchical Hybrid Sparsity Encoder:**
  - Position SRAM:** Stores Coordinate format and Bitmap format.
  - Dual-path Prep. Unit:** Contains Rulebook Gen. and Bitmap 3D Im2col.
  - Result SRAM:** Stores Format Trans. Unit (x, y, z) ↔ 01000011 and Sparsity Eval (NNZ Counter).
- Feature 2: Reconfigurable Spiking Engine:**
  - Weight SRAM:** Stores Weight #0, Weight #1, etc.
  - Spike Feature SRAM:** Stores Spike Feature #0, Spike Feature #1, etc.
  - Spiking Cores:** Multiple cores (e.g., Core #23, Core #0) performing GEMM-based Spiking and Rulebook-based Spiking.
- Feature 3: Hybrid Output Scheduler:**
  - Output/Boundary SRAM:** Stores Boundary Merge (Global Point Coords, Boundary Check, Boundary Sum) and Hybrid Format Merge (Coordinate/Bitmap, C, C/B, C/B, B, C, B/C, B, C, B/C, B, C, B/C, B).
- Feature 4: Hierarchical Hybrid Sparsity Decoder:**
  - Spike Generator:** Processes Membrane Potential Features and Comparator Group × 32.
  - Channel:** A diagram showing a sequence of spikes (1, 2, 3, 4) over time.

The architecture shows a data flow from the RISC-V CPU and DMA Controller into the Feature 1 encoder, which feeds into the Feature 2 spiking engine. The output of the spiking engine is processed by the Feature 3 scheduler and then the Feature 4 decoder, which finally outputs data to the Output Coord. DMA.

**Hierarchical Hybrid Representation System**

**3D point clouds** **Voxelization representation** **Local Tiling**

**Monolithic Representations**

1. Original Coordinate: Memory Footprint:  $NNz_2 \times 37$  bit
2. Original Bitmap: Memory Footprint: 618Billion  $\times$  1 bit

**Hierarchical Hybrid Representation**

1. Global Coordinate: Memory:  $NNz_2 \times 27$  bit
2. Global Bitmap: Memory: 61M  $\times$  1 bit
1. Local Coordinate: Memory:  $NNz_2 \times 10$  bit
2. Local Bitmap: Memory: 1024  $\times$  1 bit

them into an optimized format for efficient off-chip transmission.

### A. Hierarchical Hybrid Point Cloud Representation

3D point cloud data is characterized by two key distributional traits: extremely high sparsity due to wide-range sensing, and spatial clustering where dense clusters are scattered in expansive void space. These traits make monolithic representation inefficient for large-scale 3D point clouds. As shown at the top of Fig. 4, we take point cloud size  $(x, y, z, \text{channel}, \text{timestep}) = (1409, 1409, 121, 64, 4)$  and tile size = 4 as an illustration, with  $NNZ$  denoting the number of non-zero voxels,  $NNZ_O$ ,  $NNZ_G$ , and  $NNZ_L$  representing  $NNZ$  in the original, global-tile, and local-tile spaces,  $c$  denoting channel,  $t$  denoting timestep, and  $MF$  denoting memory footprint bits. We have:  $MF_{\text{COO}} = NNZ * (\log_2 x + \log_2 y + \log_2 z + \log_2 c + \log_2 t)$  and  $MF_{\text{Bitmap}} = x * y * z * c * t$ . With a typical  $NNZ_O = 1.5 \times 10^4$ , the memory footprint of a monolithic bitmap or coordinate representation would be enormous and unacceptable for edge applications.

To solve this problem, we aim to explore a specific representation framework utilizing hierarchy and hybrid representation format.

In terms of hierarchy, prior researches have investigated various partitioning strategies [7], [8], [9], [33]. However, these approaches primarily focused on fine-grained and uneven partitioning, which would incur complex preprocessing overhead. Specifically, tree-based partitioning methods [8], [9], [33] usually require multi-level node traversal, explicit pointer management, and irregular control flow for locating active voxels, introducing a  $O(N)$  to  $O(N^2)$  time complexity. On the other hand, hash-based block partitioning [7] achieves a time complexity of  $O(1)$ , but it introduces heavy global hash-table storage, complex collision resolution, and irregular memory accesses. In this work, we use a coarse-grained, fixed-size block-based partitioning method similar to [7] to construct a hierarchical global-local structure for hardware simplicity. Each voxel coordinate is deterministically decomposed into a global tile index and a local intra-tile offset. The global level identifies the spatial tile, while the local level describes the voxel position inside the bounded tile. While different from [7], we implement a lightweight Binary Search Finite State Machine with a  $O(\log(N))$  time complexity. It eliminates the need for complex workload schedulers, massive LUT/crossbar arrays, or large-capacity memory for redundant pointer or hash information. In this way, we achieve a balance between time complexity and hardware complexity.

After the global-local tile hierarchy is constructed, each local tile still requires an efficient internal representation. Sparse data typically adopts two addressing schemes with distinct hardware trade-offs. Coordinate-based representation explicitly uses high-precision indices to skip empty regions, but incurs severe bit-width and bandwidth overhead per voxel. Conversely, bitmap-based representation implicitly maintains spatial relationships using a 1-bit occupancy map; however, its mandatory full-volume representation creates overwhelming overhead for non-informative voids. Considering the fluctuating local sparsity in large-scale 3D environments, relying on a single monolithic representation fails to achieve optimal coding efficiency. To address this, we propose a hybrid coding scheme that dynamically switches between coordinate and bitmap representations. For a local block of size  $n \times n \times n$ , the bit-width costs for the coordinate ( $B_{COO}$ ) and bitmap ( $B_{bitmap}$ ) representations are formulated as  $B_{COO} = NNZ \times 3 \lceil \log_2(n) \rceil$  bits and  $B_{bitmap} = n^3$  bits, where  $NNZ$  denotes the number of non-zero elements in the block. Consequently, we derive a switching threshold  $T_{sw} = \lceil \frac{n^3}{3 \lceil \log_2(n) \rceil} \rceil$ . By comparing the actual  $NNZ$  against  $T_{sw}$ , our scheme adaptively selects the most memory-efficient representation for each local tile. Different from previous hybrid coding works such as [36], the threshold of switching in our work is dynamic, which allows for a more flexible and wider application range among different tasks and datasets.

### B. Adaptive Hybrid Codec Design

To facilitate the proposed hierarchical hybrid coordinate system, an adaptive hybrid codec is implemented in system level as shown in Fig. 4, integrating synergistic off-chip

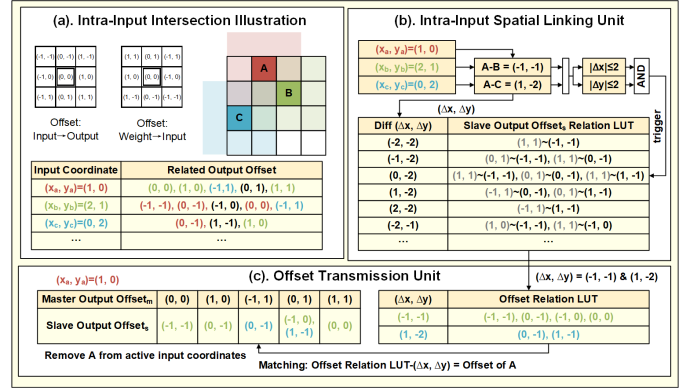


Fig. 5. Implementation of the intra-input spatial linking unit and the offset transmission unit.

and on-chip components for a seamless end-to-end hardware dataflow. The raw LiDAR input coordinates  $(x, y, z, r)$  are first preprocessed off-chip on host PC for voxelization and spike generation, and then tiled on FPGA to the most efficient format by counting local non-zero voxels and compared with pre-configured threshold register. The compact data tiles and features are then sent to the chip through a data multiplexer.

Upon on-chip reception, the data is split into dual paths for further preprocessing. Coordinates need rulebook generator to locate input-output-weight relationship, and bitmaps need im2col conversion for general matrix multiplication. The preprocessed data is then sent to the engine for computation.

We further elaborate the spatial linking and offset transmission module in rulebook generation. In conventional rulebook implementations, spatial connectivity is maintained through explicitly storing input coordinates, output coordinates, and weight offsets [14]. However, since the relationship between input, output and kernel are deterministic in convolution, the explicit representation of output constitutes redundancy. We can then simply store input coordinates and a set of offsets to represent the triplets in conventional rulebook, as shown in Fig. 5(a). For a explicit illustration, we use 2D scenario as an example.

An Intra-Input Spatial Linking (I2SL) module and an Offset Transmission (OT) module are implemented to efficiently generate these offsets. For example, voxel A is designated as the “parent” voxel, while B and C serve as “child” voxels. The I2SL module first calculates the spatial displacements between the parent and child voxels (e.g.,  $diff = A - B$ ). If the displacement in both  $x$  and  $y$  dimensions remains within a threshold of two (for a  $3 \times 3$  sparse convolution kernel), a spatial intersection is confirmed, triggering a lookup in a pre-configured child output offset relation lookup table (LUT) via the AND result signal. Meanwhile, the displacement vector  $diff = (\Delta x, \Delta y)$  is sent to the LUT as the index to retrieve the valid child offsets from the Relation LUT, as shown in Fig. 5(b). The LUT provides the child offsets ( $offset_s$ , marked in black); notably, the corresponding parent offsets ( $offset_m$ , marked in gray) are not explicitly stored, as they can be implicitly derived via the hardware relation:  $offset_m = offset_s - diff$ . The OT module then expands each child-

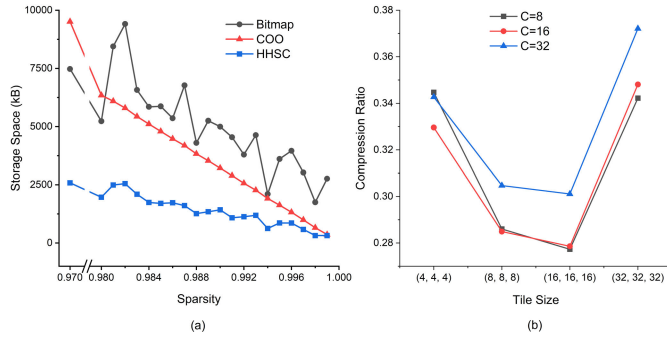


Fig. 6. Evaluation result on (a) the storage space of different formats across sparsity and (b) different tile sizes' impact on data compression ratio.

centered offset into the parent-centered coordinate system by the relation mentioned above, and groups the parent-child offset pairs according to the derived  $offset_m$ , as illustrated in Fig. 5(c). This table generated by OT allows the downstream rulebook engine to directly obtain the corresponding weight based on the offsets using the output-weight relationship. After all slots of the current parent voxel are processed, the parent voxel is removed from the active list while the child voxels are retained, avoiding redundant pair generation in subsequent windows.

For the decoder stage, computed data is streamed from the output scheduler in hybrid formats. Spike conversion is then applied to generate discrete 1-bit data for data compression. Before sending off-chip, the spike tiles will pass a sparsity evaluation module to further check the efficiency of current storage format by non-zero counting. If not efficient, a format transcoding unit can switch the format to secure that the data stays compressed. The checked spike tiles are then sent off-chip and prepared to be refilled into local bitmap or coordinate buffers for the next iteration.

### C. Evaluation

We quantify the contribution of the proposed hierarchical hybrid coding scheme to external memory access (EMA), as shown in Fig. 6(a). Since the on-off chip interface operates with a deterministic data transmission rate, EMA is proportional to the compressed storage footprint, i.e.,  $EMA \propto \frac{\text{Storage Space}}{\text{Data Transmission Rate}}$ . Therefore, the EMA reduction achieved by our coding scheme directly translates into a proportional reduction in storage space. We only show the sparsity scale from 0.97 to 0.999 since this is the typical sparsity range of 3D backbone in all of our tasks. Under such sparsity, the global coordinate will surely outperform bitmap, thus we directly set the global format as coordinate-based in this experiment and show the difference of local format here. As shown in the figure, the coordinate shows a linear relationship with sparsity due to fixed-size tiling. When the sparsity level is 0.980 or higher, coordinate will always outperform bitmap since there are few valid voxels to store. And due to the spatial clustering and fine-grained tile-level sparsity exploration, HHSC will always outperform both bitmap and coordinate format.

We have also discussed the influence of tile size on the data compression ratio, as shown in Fig. 6(b). The compression

ratio increases for both small and large tiles for different reasons. For small tiles, the decrease of shared global coordinates limits the utilization of hierarchical scheme. For large tiles, the spatial clustering feature becomes diluted, which weakens the advantages of hybrid coding. This principle similarly applies to the channel dimension, treated here as the fourth dimension of the tiled data. While a tile size of (16, 16, 16, 8) achieves the theoretical optimal compression, we adopt a (8, 8, 8, 16) configuration in this work to prioritize non-zero counting efficiency for hardware processing.

## V. RECONFIGURABLE SPIKING ENGINE

To accommodate the divergent dataflow generated by the hierarchical hybrid codec and ensure low data transmission to off-chip, a set of reconfigurable spiking cores is proposed as the primary computational engine. Given that coordinate-based and bitmap-based representations necessitate fundamentally different operational patterns, a monolithic compute architecture would inevitably incur significant hardware under-utilization and throughput bottlenecks. Specifically, while the coordinate path requires fine-grained serial logic, the bitmap path is optimized for coarse-grained, high-parallelism execution. Besides, intermediate partial sums in 3D sparse convolutions typically exhibit high bit-widths, which would incur large metadata transmission if not optimized. The integration of neuromorphic activation functions provides an energy-efficient quantization mechanism. By mimicking biological integrate-and-fire dynamics, this approach can compress high-precision accumulated data into 1-bit discrete spikes, effectively suppressing the power consumption associated with massive data shuttling.

### A. Reconfigurable Spiking Core Design

The overall architecture of the proposed reconfigurable fused spiking core is illustrated in Fig. 7. For rulebook-based computation, input coordinates and their associated offset groups are organized in the rulebook buffer. Each offset group begins with a primary parent offset followed by a sequence of child offsets, all of which are dispatched to a specialized weight-offset transformation module. The hardware derives the necessary weight indices via a negation operation (i.e.,  $W_{off} = -O_{off}$ ), enabling low-latency indexing into the weight buffer. The retrieved synaptic weights are subsequently buffered in a queue register to feed a serial adder for accumulation. Finally, the output coordinates and accumulated partial sums are stored in the output buffer or boundary buffer according to the coordinate range. For the GEMM-based computation, inputs are rearranged into a  $(K^3 \cdot C_{in})$  1-bit spike matrix to facilitate dense matrix-vector operations. These 1-bit signals are broadcast to serve as gating controls for multiplexing weights, which effectively eliminates the necessity for power-intensive multipliers, significantly enhancing energy efficiency.

In terms of the reconfigurable engine, a baseline of 16 independent adder trees is implemented. In each adder tree, we implemented seven of the adders in the normal way, while another eight adders are lifted to 11b adders. Such design allows the engine to be reconfigured into two modes, as

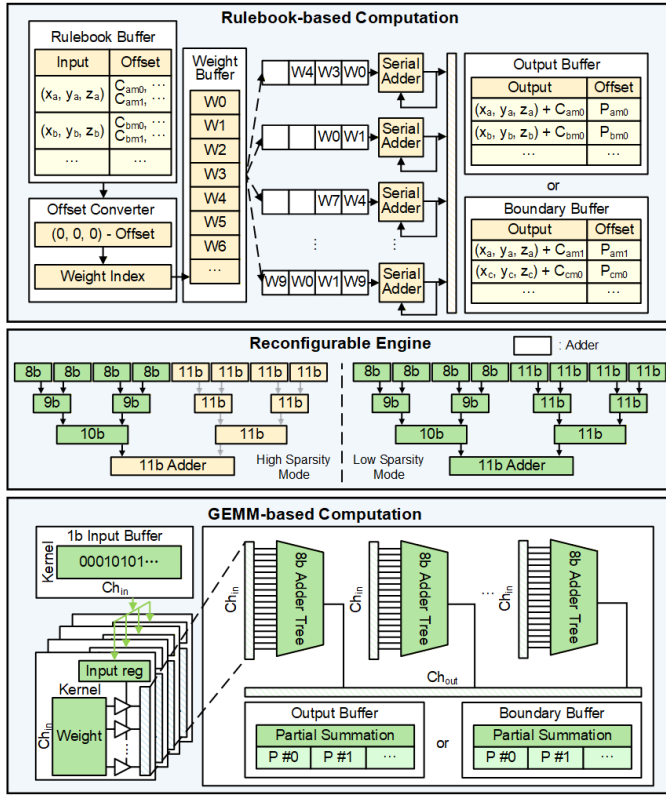


Fig. 7. Illustration of the reconfigurable spiking core.

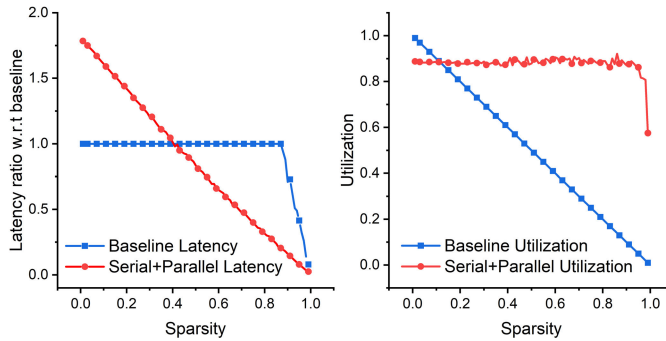


Fig. 8. The latency and utilization of two configured modes across different sparsity levels.

shown in the middle of Fig. 7. When the sparsity level is low, bitmap format would dominate the data pipeline, and the engine will be configured into 16 8-bit 16-way fan-in adder trees, with 1 to 3 bit unused in 11b adders. When the sparsity level gets high and serial adders are needed, the 11b adders will be separated from the original adder tree and serve as 8 independent 11b serial adders (marked in yellow). The left half of the original adder tree (marked in green) will serve as a 8-bit 8-way fan-in adder tree, still providing half of the computing power as it used to be. Such design would incur 18.9% overhead, while fully release the power of sparse rulebook-based computation. As shown in Fig. 8, both the latency and utilization shows a huge promotion in high sparsity range (from 0.8-0.99). The drops in both curves are due to the high sparsity incurred empty tiles, which is skipped by our

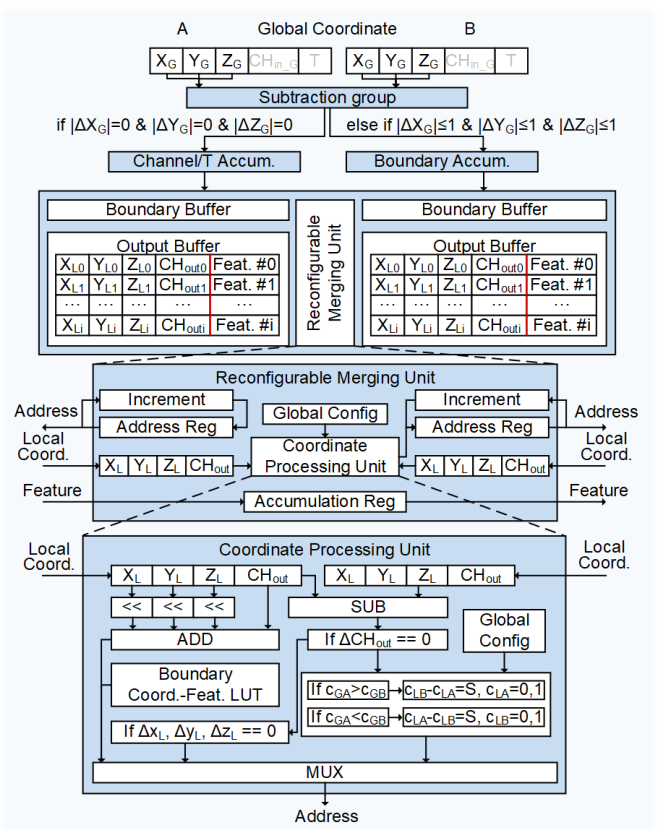


Fig. 9. Illustration of the hybrid output scheduler.

codec. We evaluated the reconfigurable engine's performance across varying sparsity ratios, as this metric fundamentally dictates the dynamic proportion of coordinate versus bitmap formats. In our hierarchical hybrid codec, the number of non-zero elements within a local tile determines the chosen data representation. Therefore, sweeping the sparsity ratio effectively sweeps the format mixture, comprehensively validating the engine across diverse dynamic data distributions.

For potential pipeline stalls caused by the reconfiguration among cores, we utilize software scheduling to balance the synchronization bubbles. Based on the computing mode and sparsity level, the compiler distributes varied channel workloads across different cores, achieving a balanced latency.

### B. Hybrid Output Scheduler

Following the generation of output partial sums by the reconfigurable compute cores, an efficient scheduling mechanism is essential to orchestrate online accumulation and ensure the integrity of the final membrane potentials prior to spike generation. This process encompasses two primary levels of data merging: hybrid-format accumulation across channels and timesteps, and spatial boundary synchronization between adjacent tiles. To address these requirements, a hybrid output scheduler is proposed, as illustrated in Fig. 9. The hardware initiates processing by performing a spatial proximity analysis on the global coordinates ( $x_G, y_G, z_G$ ) of candidate output tiles. Identical global coordinates trigger accumulation within the

channel or temporal dimensions, whereas a global displacement within a unit voxel ( $\Delta \leq 1$ ) signifies a spatial boundary intersection requiring consistency across the tile frontiers.

Central to this architecture is a Reconfigurable Merging Unit (RMU), which adaptively manages three distinct dataflow scenarios based on format mode IDs: coordinate-plus-bitmap (COO+BMP), bitmap-plus-bitmap (BMP+BMP), and coordinate-plus-coordinate (COO+COO). For the COO+BMP case, local coordinates are retrieved from the output/boundary buffer and translated into bitmap feature addresses; while intra-tile addresses are derived using hardware-efficient shifters and adders, boundary coordinates are resolved via a specialized Boundary Coordinate-Feature Lookup Table. In the BMP+BMP mode, the structural alignment of binary maps allows for synchronized address incrementers to fetch and accumulate features. For COO+COO merging, the hardware performs subtractive coordinate analysis to verify channel alignment and local spatial proximity ( $\Delta X_L, \Delta Y_L, \Delta Z_L$ ), enabling direct feature updates for matched indices. The boundary detection logic synergistically evaluates global and local constraints; for instance, when a global offset  $\Delta C_G, C = X, Y, Z$  is detected, the system applies local constraints ( $\Delta C_L = S, C_{L,min} = 0, 1, C = X, Y, Z$ ) to accurately target and merge overlapping features ( $S = \text{tile size} - 2 * \text{stride}$ ). Finally, a multiplexing network selects the generated addresses to fetch corresponding feature data from the buffers, completing the end-to-end accumulation cycle.

For the output membrane potential update across timesteps, our system ensures a minor SRAM access overhead of around 2% thanks to our short timesteps in algorithm and the exploitation of inherent sparsity in coordinate-format output potentials. When the channel depth scales up in the deeper layers of the 3D backbone, our software configurations can make sure that the scheduler does not become the system bottleneck by adjusting tile sizes to alleviate the latency issue.

## VI. IMPLEMENTATION RESULTS AND COMPARISON

### A. Fabrication Result

The proposed chip SHARP was fabricated using 28-nm CMOS technology. Fig. 10 illustrates the die micrograph along with the detailed chip specifications. The total die area, including the I/O pads, is  $2.75 \text{ mm} \times 3.3 \text{ mm}$ , resulting in an overall silicon footprint of  $9.075 \text{ mm}^2$ . Measurement results demonstrate that the prototype chip is capable of operating across a frequency range from 25 MHz to 200 MHz, with the supply voltage scaling from 0.55 V to 1.1 V. The chip achieves its peak energy efficiency of 83.24 TOPS/W at the voltage supply of 0.55 V with an operating frequency of 25 MHz. Error Correction Code (ECC) is introduced to mitigate the unreliability of SRAM under extreme conditions. The reported energy efficiency strictly accounts for the power consumption of the SHARP accelerator chip itself. It excludes the power from the FPGA, external DRAM, and communication interfaces. This scope was chosen to align with the standard evaluation methodology adopted in closely related state-of-the-art works [7], [8], thereby ensuring a fair and consistent baseline for comparing the core hardware architectures.

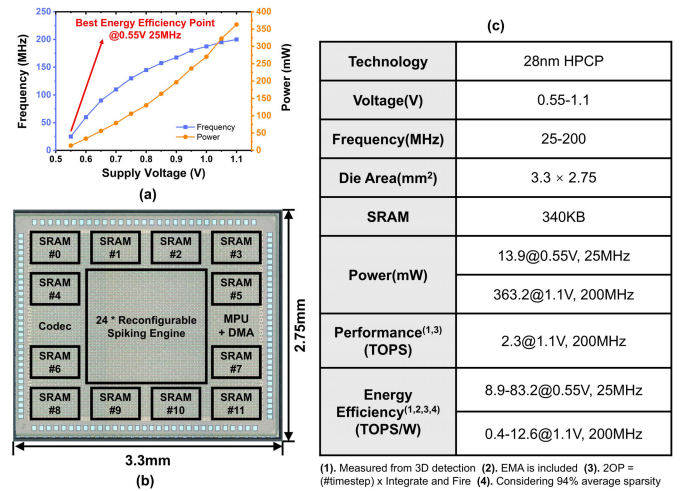


Fig. 10. (a) The voltage frequency scaling map. (b) The die micrograph of SHARP. (c) The specification of SHARP.

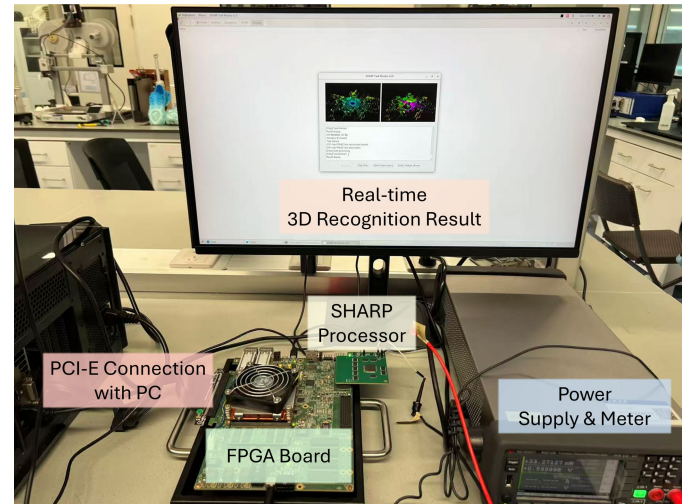


Fig. 11. Photograph of the chip testing platform.

The chip testing platform is shown in Fig. 11. The accelerator is mounted on an FPGA board (Xilinx Virtex UltraScale+) to communicate with the host PC. The screen shows the test results on semantic-kitti dataset for 3D semantic segmentation task, including input point clouds, output predicted labelled point clouds and running logs. The accelerator has an independent power supply from a Keysight B2902A DC power analyzer, which measures the core's power consumption from 13.96 mW to 363.25 mW. The preprocessing stage, if implemented on SoC, can be deployed on a low-power DSP, which would estimate to consume around 0.825 mW energy consumption and 7.5 ms latency minimum [37], incurring minor overhead to the system.

### B. Task Performance

The performance of SHARP is evaluated across several 3D recognition benchmarks, including classification, segmentation, and detection. All models are pretrained on an NVIDIA A800 GPU, adopting the network architectures and configurations from [25]. The detailed results are summarized in Table I.

TABLE I  
SHARP'S 3D RECOGNITION TASK PERFORMANCE

| Task Performance Table                    |                            |                 |                                    |
|-------------------------------------------|----------------------------|-----------------|------------------------------------|
| Task                                      | 3D Classification          | 3D Segmentation | 3D Detection                       |
| Accuracy/mIoU/mAP (%)                     | 90.36                      | 67.76           | 88.91, 83.32, 77.93 <sup>(1)</sup> |
| Accuracy/mIoU/mAP Loss (%) <sup>(2)</sup> | 1.34                       | 0.74            | 0.69, 0.68, 0.77 <sup>(1)</sup>    |
| Dataset                                   | ModelNet40                 | Semantic-KITTI  | KITTI                              |
| No. of Parameters                         | 3.27M                      | 17.9M           | 8.5M                               |
| Input Point Cloud Scale <sup>(3)</sup>    | 247×144×283 <sup>(4)</sup> | 1409×1409×121   | 1408×1600×41                       |
| EMA reduction (%)                         | 79.8%                      | 72.5%           | 56.1%                              |
| Frame Latency (ms) <sup>(5)</sup>         | 1.01                       | 8.34            | 22.27                              |
| Frame Energy (mJ/frame) <sup>(6)</sup>    | 0.13                       | 1.46            | 2.25                               |

(1). Car 3D AP (R11) Easy, Moderate and Hard Score (2). Compared to results in [24]. (3). Number of voxels as the measurement unit. (4). Data from Airplane sample (5). Measured at 200MHz, 1.1V, w/ EMA (6). Measured at 25MHz, 0.55V, w/ EMA

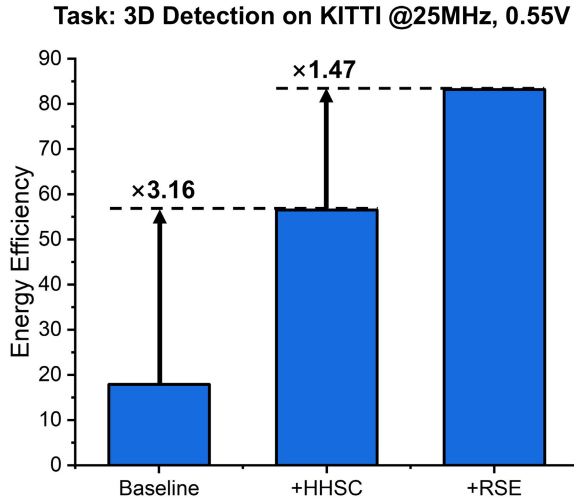


Fig. 12. The ablation study.

Specifically, SHARP achieves 90.36% accuracy on the ModelNet40 dataset [29], 67.76% mIoU on the Semantic KITTI dataset [38], and 88.91%, 83.32%, 77.93% mAP on the KITTI dataset Car 3D AP (R11) Easy, Moderate, Hard tasks [39]. Compared to the software-based algorithmic results in [25], SHARP incurs marginal performance degradations of only 1.34% (accuracy), 0.74% (mIoU), and 0.69% (easy mAP), 0.68% (moderate mAP), 0.77% (hard mAP), respectively. Furthermore, compared to a baseline dense SNN engine, SHARP achieves significant reductions in Effective Memory Access (EMA) of 79.8%, 72.5%, and 56.1% across the three tasks. Both frame latency and energy consumption measurements are reported under consideration of the EMA impacts.

We have further done the ablation study to show the effectiveness of our proposed techniques in this work, as shown in Fig. 12. By utilizing the hierarchical hybrid sparsity codec to compress the sparse point cloud data, SHARP has gained 3.16x energy efficiency boost compared to the baseline with no techniques added. Then, by further applying reconfigurable spiking engine to boost the computation, SHARP can gain another 1.47x energy efficiency, and comes to a peak energy efficiency of 83.24 TOPS/W, as reported above. The ablation study is done on 3D detection task on KITTI dataset, at 25MHz and 0.55V condition.

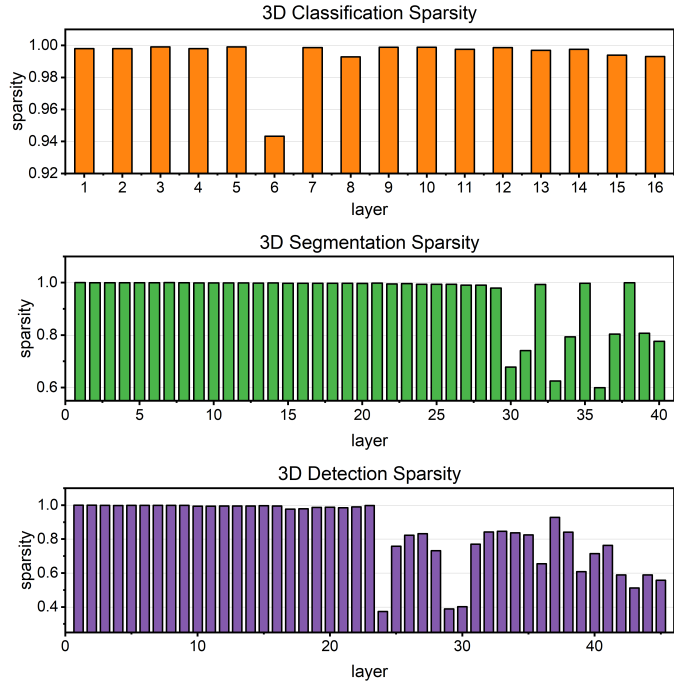


Fig. 13. The sparsity distribution of three 3D recognition tasks.

### C. Network Sparsity

The sparsity distributions across three 3D recognition benchmarks are illustrated in Fig. 13. Our analysis focuses exclusively on the spiking layers and the initial stem layers for spike generation are omitted from this summary as they are deployed in off-chip Hierarchical Hybrid Sparsity Codec (HHSC). The average sparsity of the 3D backbone is exceptionally high, exceeding 99%, which highlights the efficacy of the proposed HHSC in processing these sparse layers. Conversely, for the 2D backbone and linear layers where sparsity is relatively lower, the Reconfigurable Spiking Engine (RSE) maintains high hardware utilization and low computational latency with minimal overhead. Furthermore, as the feature dimensionality decreases in these downstream layers, the overall data volume remains relatively small, ensuring that the EMA does not significantly increase despite the lower sparsity.

### D. Programmability

To accommodate diverse 3D SNN architectures, varying kernel geometries, channel dimensions, and time steps, SHARP employs a hardware-software co-design for dynamic runtime reconfiguration. Due to limited on-chip storage, the system utilizes multi-layer nested loops to process large workloads iteratively. During each iteration, the host RISC-V CPU executes core C++ operators to orchestrate data streaming and hardware adaptation. The compiler translates these operators into unified instructions that write global variables—encompassing off-chip memory addresses, data packet dimensions, core control parameters, programmable spatial offsets ( $\Delta x, \Delta y$ ), and updated Relation LUT rules—directly into SHARP's configuration registers via the

TABLE II  
COMPARISON WITH STATE-OF-THE-ART 3D POINT CLOUD ACCELERATORS

| Performance Comparison Table                      |                            |                           |                               |                               |                                |                                                                                               |
|---------------------------------------------------|----------------------------|---------------------------|-------------------------------|-------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------|
|                                                   | JSSC'24 [7]<br>Feng et al. | JSSC'24 [8]<br>Lim et al. | TCAS-I'25 [13]<br>Zhao et al. | TCAS-II'25 [34]<br>Lee et al. | TCAS-I'26 [11]<br>Seong et al. | This work                                                                                     |
| Tech(nm)                                          | 28                         | 28                        | 22                            | 28                            | 28                             | 28                                                                                            |
| Application                                       | 3D Detection               | 3D Detection              | 3D Detection                  | 3D Cls.+Seg.*                 | 3D Detection                   | 3D Recognition<br>(Cls.+Det.+Seg.)                                                            |
| Method                                            | Voxel-based                | Pillar-based              | Voxel-based                   | Spiking<br>Point-based        | Point-based                    | Spiking<br>Voxel-based                                                                        |
| Supply Voltage (V)                                | 0.48-0.92                  | 0.74-1.1                  | 0.8                           | 0.9                           | 0.9                            | 0.55-1.1                                                                                      |
| Frequency (MHz)                                   | 40-450                     | 320                       | 400                           | 200                           | 200                            | 25-200                                                                                        |
| Die Area(mm <sup>2</sup> )                        | 3.24                       | 12.96                     | 1.82                          | 4.16                          | 3.24                           | 9.075                                                                                         |
| Power (mW)                                        | 6.6-179.4                  | 66.1-855.5                | 137.79                        | 9.0                           | 195.6 @0.9V                    | 13.9-363.2                                                                                    |
| On-Chip SRAM (kB)                                 | 176                        | 741                       | 231.3                         | 832                           | 431                            | 340                                                                                           |
| Precision                                         | INT8                       | INT5/9/13/17              | INT8                          | INT1/6(A)+<br>INT6(W)         | INT8                           | INT4(A) <sup>(1)</sup> +<br>INT8(W)                                                           |
| Peak Energy Efficiency <sup>(2)</sup><br>(TOPS/W) | 4.14                       | 38.50                     | 7.79                          | 23.12 (Seg.)                  | -                              | 65.72 (Seg.) <sup>(3,4)</sup><br>83.24 (Det.) <sup>(3,4)</sup>                                |
| Energy Consumption (mJ/frame)                     | -                          | -                         | 3.9                           | -                             | 2.57                           | 0.13 (Cls.) <sup>(3,4)</sup><br>1.46 (Seg.) <sup>(3,4)</sup><br>2.25 (Det.) <sup>(3,4)</sup>  |
| Latency (ms)                                      | 59.2                       | 22.6 <sup>(3)</sup>       | 28.4                          | -                             | 12.8                           | 1.01 (Cls.) <sup>(3,5)</sup><br>8.34 (Seg.) <sup>(3,5)</sup><br>22.27 (Det.) <sup>(3,5)</sup> |

(1). 1b spikes with 4 timesteps (2). For SNN, 2OP=1MAC= (8 timesteps) x Integrate and Fire (3). EMA is included (4). @25MHz, 0.55V (5). @200MHz, 1.1V \* Cls.: Classification, Seg.: Segmentation, Det.: Detection

system bus. The configuration registers are a set of ping pong registers to hide the configuration latency into computation latency. Quantitatively, The total configuration latency is approximately 420 cycles among layers, whereas the minimum computation latency is approximately 5000 cycles among layers. With ping pong registers, the CPU-to-accelerator communication can be pipelined with accelerator computation and would not contribute noticeable overhead to the total inference latency. Once initialized, the on-chip Direct Memory Access (DMA) module autonomously fetches the specified data. By adjusting outer loop iteration counts and continually updating these registers, the system scales to arbitrary network sizes and redefines the connection topology with zero hardware overhead.

Driven by this dynamically configured framework, SHARP's reconfigurable engine supports all mainstream network layers within a unified datapath. For sparse operations, the software-programmable Relation LUT functions as a rulebook indexing paradigm for mapping input-output coordinate pairs. In a standard sparse convolution (SConv) with kernel size  $K$ , spatial offsets are configured to  $|\Delta x| \leq K - 1$  and  $|\Delta y| \leq K - 1$ , allowing new active values in overlapping regions by mapping multi-pairing relationships in the LUT (Fig. 5(b)). Conversely, for submanifold 3D convolutions (SubMConv3d), where activations are generated only at non-zero center points, offsets are bounded to  $|\Delta x| \leq \lceil \frac{K}{2} \rceil$  and  $|\Delta y| \leq \lceil \frac{K}{2} \rceil$ . The CPU subsequently modifies the LUT to eliminate dilated mappings; for example, the relative spatial row  $(\Delta x, \Delta y) = (-1, -1)$  is restricted from multiple overlaps to a single  $(-1, -1)$  pairing. Beyond sparse processing, the accelerator accommodates dense topologies.

Standard 2D convolutions (Conv2d) are automatically identified, triggering a switch to a bitmap encoding mode where workloads are routed to dynamically partitioned GEMM cores for high-utilization acceleration. Similarly, fully-connected or linear layers degenerate into channel-wise matrix multiplications, which are directly allocated to these GEMM cores to maximize overall computing throughput.

#### E. Comparison With State-of-the-Art Accelerators

We then compare SHARP with state-of-the-art 3D point cloud processors across various tasks, as shown in Table II. Since we adopt SNN, a new neuromorphic paradigm in this work, to make a fair comparison across SNN and non-SNN works, we first need to define the energy efficiency in SNN. According to the convention in [15], [40], [41], and [42], we define 2OPs to represent spike accumulations of 8 bit weights across 8 timesteps. This normalization is physically grounded in the fact that, under 28nm CMOS technology, the energy consumption of one 8-bit multiplier is approximately 8 times that of an 8-bit adder, as we experimented in synthesis level. We normalized two SNN works, [34] and our work for a rigorous comparison with other non-SNN works.

In terms of energy efficiency, SHARP achieves a  $2.16\times$  improvement over the state-of-the-art (SOTA) point cloud accelerator [8] on the 3D detection task, and a  $2.84\times$  improvement over the SOTA SNN-based point cloud accelerator [34] on the 3D segmentation task, primarily owing to the proposed hierarchical hybrid coding and hardware optimization techniques. We further evaluate the energy consumption in the task level for a more thorough comparison. SHARP has consumed 2.25 mJ/frame for 3D detection task on KITTI

dataset. For comparison, our work achieves a 12% reduction in task-level energy consumption compared to the best-known prior work [11].

Regarding latency, our 3D detection task reports 22.27 ms, which is also quite competitive in state-of-the-art works, as shown in Table II. This latency is primarily attributed to the lower sparsity levels encountered in the deep 2D backbone of the detection network. However, for classification and segmentation tasks where the 3D backbone dominates, SHARP achieves latencies of 1.01 ms and 8.34 ms, respectively. These results, which account for full EMA impacts, demonstrably satisfy the real-time requirements for edge deployment.

## VII. CONCLUSION

In conclusion, this paper presents SHARP, a spiking reconfigurable accelerator optimized for the energy-efficient deployment of 3D point cloud recognition tasks. By integrating a hierarchical hybrid sparsity codec to minimize memory overhead with a rulebook-GEMM fused engine to maximize PE utilization, SHARP delivers a significant breakthrough in power-efficient 3D perception. Fabricated in 28 nm CMOS, the prototype chip achieves the lowest task energy consumption and a peak efficiency of 83.24 TOPS/W, which is a 2.16x boost compared to state-of-the-art processors. SHARP establishes a robust architectural framework for high-performance 3D recognition at the edge, providing a finely optimized balance of power, latency, and throughput.

## REFERENCES

- [1] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, and W. Shi, "Edge computing for autonomous driving: Opportunities and challenges," *Proc. IEEE*, vol. 107, no. 8, pp. 1697–1716, Aug. 2019.
- [2] L. Liu et al., "Computing systems for autonomous driving: State of the art and challenges," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6469–6486, Apr. 2021.
- [3] C. Zhang, Y. Zhang, J. Yu, J. Yang, and M. Sawan, "Event-fused hybrid ANN-SNN architecture for low-latency object detection in automotive vision," *IEEE Robot. Autom. Lett.*, vol. 11, no. 3, pp. 3622–3628, Mar. 2026.
- [4] L. Yan et al., "Robotic computing system and embodied AI evolution: An algorithm-hardware co-design perspective," *J. Semiconductors*, vol. 46, no. 10, Oct. 2025, Art. no. 101201.
- [5] S. Kim, J. Lee, D. Im, and H.-J. Yoo, "PNNPU: A 11.9 TOPS/W high-speed 3D point cloud-based neural network processor with block-based point processing for regular DRAM access," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.
- [6] Q. Cao and J. Gu, "A sparse convolution neural network accelerator for 3D/4D point-cloud image recognition on low power mobile device with hopping-index rule book for efficient coordinate management," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 106–107.
- [7] X. Feng et al., "A 28-nm energy-efficient sparse neural network processor for point cloud applications using block-wise online neighbor searching," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3070–3081, Sep. 2024.
- [8] S. Lim, J. Heo, J. Yang, and J.-Y. Kim, "Hawkeye: A point cloud neural network processor with virtual pillar and quadtree-based workload management for real-time outdoor BEV detection," *IEEE J. Solid-State Circuits*, vol. 60, no. 3, pp. 990–1001, Mar. 2025.
- [9] C. Zhou et al., "23.4 Nebula: A 28nm 109.8TOPS/W 3D PNN accelerator featuring adaptive partition, multi-skipping, and block-wise aggregation," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2025, pp. 412–414.
- [10] B. Seo, J. Jung, D. Han, and K. J. Lee, "A LiDAR-PNN pipelined processor with cylindrical bin partitioning and halo indexing for 3-D perception in outdoor autonomous driving applications," *IEEE J. Solid-State Circuits*, vol. 60, no. 10, pp. 3655–3664, Oct. 2025.
- [11] C. Seong, S. Lee, B. Kim, and D. Jeon, "An area- and energy-efficient point-based 3-D object detection processor with feature-decoupled PointNet for autonomous driving," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 73, no. 8, pp. 5487–5500, Aug. 2026.
- [12] B. Dally, "Hardware for deep learning," in *Proc. IEEE Hot Chips Symp. (HCS)*, Aug. 2023, pp. 1–58.
- [13] P. Zhao et al., "QSAF: Energy and area-efficient query-based sparsity-aware accelerator for voxel-based point cloud neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 73, no. 5, pp. 3336–3349, May 2026.
- [14] B. Graham, M. Engelcke, and L. van der Maaten, "3D semantic segmentation with submanifold sparse convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9224–9232.
- [15] C. Fang et al., "An energy-efficient unstructured sparsity-aware deep SNN accelerator with 3-D computation array," *IEEE J. Solid-State Circuits*, vol. 60, no. 3, pp. 977–989, Mar. 2025.
- [16] X. Cheng, S. Cao, S. Wang, M. Wang, W. Li, and X. Zeng, "An FPGA-based event-driven SNN accelerator for DVS applications with structured sparsity and early-stop," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 7, pp. 3298–3310, Jul. 2025.
- [17] H. Zhang, S. Zhang, W. Mao, and Z. Wang, "An energy-efficient neuromorphic accelerator based on deformable spiking transformer for dynamic vision sensor," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 12, pp. 7860–7873, Dec. 2025.
- [18] C. Sun et al., "A 1.1  $\mu$ J/inference binary spiking neural network accelerator for DVS gesture recognition," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 73, no. 2, pp. 1093–1106, Feb. 2026.
- [19] M. Li, Y. Kan, M. Wu, R. Zhang, and Y. Nakashima, "FPSpike: A fully parallel and reconfigurable architecture for accelerating spiking neural networks with structured sparsity," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 73, no. 5, pp. 3546–3559, May 2026.
- [20] D. Ren et al., "Spiking PointNet: Spiking neural networks for point clouds," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 41797–41808.
- [21] Y. Lan, Y. Zhang, X. Ma, Y. Qu, and Y. Fu, "Efficient converted spiking neural network for 3D and 2D classification," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 9177–9186.
- [22] Q. Wu, Q. Zhang, C. Tan, Y. Zhou, and C. Sun, "Point-to-spike residual learning for energy-efficient 3D point cloud classification," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 6, pp. 6092–6099.
- [23] P. Wu et al., "Efficient spiking point Mamba for point cloud analysis," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2025, pp. 26393–26403.
- [24] P. Wu et al., "Spiking point transformer for point cloud classification," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 20, pp. 21563–21571.
- [25] X. Qiu et al., "Efficient 3D recognition with event-driven spike sparse convolution," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 19, pp. 20086–20094.
- [26] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 652–660.
- [27] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–10.
- [28] G. Qian et al., "PointNeXt: Revisiting PointNet++ with improved training and scaling strategies," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 23192–23204.
- [29] Z. Wu et al., "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1912–1920.
- [30] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4490–4499.
- [31] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, Oct. 2018.
- [32] J. Deng et al., "Voxel R-CNN: Towards high performance voxel-based 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 1201–1209.
- [33] D. Lyu, Z. Li, Y. Chen, J. Zhang, N. Xu, and G. He, "SpOctA: A 3D sparse convolution accelerator with octree-encoding-based map search and inherent sparsity-aware processing," 2023, *arXiv:2308.09249*.
- [34] S. Lee et al., "A 227.1 TOPS/W high energy-efficiency PNN-based 3-D object recognition processor with spiking neural network for edge device," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 11, pp. 1610–1614, Nov. 2025.

- [35] W. Wang et al., "An FPGA processor combining point cloud and SNN for DVS-based ADAS application," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2025, pp. 1–5.
- [36] C. Li, S. Li, Y. Zhao, W. Zhu, and Y. Lin, "RT-NeRF: Real-time on-device neural radiance fields towards immersive AR/VR rendering," in *Proc. 41st IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Oct. 2022, pp. 1–9.
- [37] NXP Semiconductors. (Jul. 2022). *Using IMX RT500 FusionF1 DSP Low-Power*. [Online]. Available: <https://www.nxp.com/docs/en/application-note/AN13657.pdf>
- [38] J. Behley et al., "SemanticKITTI: A dataset for semantic scene understanding of LiDAR sequences," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9296–9306.
- [39] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361.
- [40] S. Kim, S. Kim, S. Um, S. Kim, J. Lee, and H.-J. Yoo, "SNPU: An energy-efficient spike domain deep-neural-network processor with two-step spike encoding and shift-and-accumulation unit," *IEEE J. Solid-State Circuits*, vol. 58, no. 10, pp. 2812–2825, Oct. 2023.
- [41] S. Kim, S. Kim, S. Hong, S. Kim, D. Han, and H.-J. Yoo, "C-DNN: A 24.5–85.8TOPS/W complementary-deep-neural-network processor with heterogeneous CNN/SNN core architecture and forward-gradient-based sparsity generation," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2023, pp. 334–336.
- [42] S. Kim, W. Jo, and S. Hong, "C-Transformer: A 2.6–18.1  $\mu$ J/token homogeneous DNN-transformer/spiking-transformer processor with big-little network and implicit weight generation for large language models," in *Proc. ISSCC*, 2024, pp. 368–369.



**Ziyang Shen** (Graduate Student Member, IEEE) received the B.Eng. degree in microelectronics from Fudan University, Shanghai, China, in 2022. He is currently pursuing the Ph.D. degree with the CenBRAIN Neurotech Center of Excellence, Westlake University, Hangzhou, China. His research interests include spike-based neuromorphic vision algorithm and domain-specific neuromorphic accelerator design for edge applications.



architectures.

**Zhipeng Liao** (Student Member, IEEE) received the bachelor's degree in microelectronics science and engineering from the South China University of Technology in 2023 and the parent's degree in electronic engineering from the University of Southampton in 2024. In October 2024, he joined Westlake University and is currently a Research Assistant at the CenBRAIN Neurotech Center of Excellence, School of Engineering. His main research interests lie in brain-inspired computing paradigms, neuromorphic chips, and high-performance computing



**Sitan Shen** (Student Member, IEEE) received the B.Eng. degree in microelectronics science and engineering from Tongji University, Shanghai, China, in 2024. He is currently pursuing the Ph.D. degree with the CenBRAIN Neurotech Center of Excellence, School of Engineering, Westlake University, Hangzhou, China. His research interests include neuromorphic digital chip design for embodied intelligence, sparse computing architectures, hardware-software co-design for neuromorphic systems, and domain-specific processor design for edge applications.



for deep learning accelerator, and domain-specific processor design for edge applications.



research interests lie in neuromorphic computing, neural interfaces, and computing-in-memory systems. He was a recipient of several awards, including the Best Paper Award at IEEE AICAS 2023.



he is currently a Research Professor. His research interests include integrated circuits for brain-computer interfaces, energy-efficient neuromorphic computing, and brain-language decoding.



**Mohamad Sawan** (Life Fellow, IEEE) received the Ph.D. degree from the University of Sherbrooke, Canada. He is currently a Chair Professor with Westlake University, Hangzhou, China, and an Emeritus Professor with the Polytechnique Montreal, Canada. He is also the Founder and the Director of the Center of Excellence in Biomedical Research on Advances-on-chips Neurotechnologies (CenBRAIN Neurotech), Westlake University, and the Polystim Neurotech Laboratory, Polytechnique Montreal. He worked as a Distinguished Lecturer of both CASS and SSCS for three consecutive years each. He was the Co-Founder, an Associate Editor, and the Editor-in-Chief of IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS (2016–2019). He was awarded the Canada Research Chair in Smart Medical Devices (2001–2015) and was leading the Microsystems Strategic Alliance of Quebec (ReSMiQ), Canada (1999–2018). He has published more than 1000 peer-reviewed papers, one handbook, three books, 13 book chapters, 12 patents, and 20 other patents are pending. He received several awards, among them the Barbara Turnbull Award from the Canadian Institutes of Health Research (CIHR), the J. A. Bombardier and Jacques-Rousseau Awards from Canadian ACFAS, the Queen Elizabeth II Golden Jubilee Medal, the Medal of Merit from the President of Lebanon, and Shanghai International Collaboration Award. He is a fellow of the Royal Society of Sciences of Canada (FRSC), Canadian Academy of Engineering (FCAE), the Engineering Institutes of Canada (FEIC), and Asia-Pacific Artificial Intelligence Association (FAAIA); and an "Officer" of the National Order of Quebec.

**Chaoming Fang** (Student Member, IEEE) received the B.S. degree in instrumentation science and technology from Beihang University, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with Westlake University, Hangzhou, China. He joined the Center of Excellence in Biomedical Research on Advanced Integrated-on-chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, in 2021. His current research interests include hardware-software co-design for neuromorphic computing, VLSI design

**Fengshi Tian** (Graduate Student Member, IEEE) received the B.Eng. degree from Fudan University, Shanghai, China, in 2021. He is currently pursuing the joint Ph.D. degree with the ECE Department and AI Chip Center for Emerging Smart System (ACCESS), The Hong Kong University of Science and Technology (HKUST), supervised by Prof. Kwang-Ting Tim Cheng and Prof. Chi-Ying Tsui. He was with CenBRAIN Neurotech, Westlake University, as a Visiting Student from 2020 to 2021, supervised by Prof. Mohamad Sawan. His current

**Jie Yang** (Senior Member, IEEE) received the B.S. degree in electronic science and technology from Tianjin University, Tianjin, China, in 2010, and the Ph.D. degree in microelectronics from the Institute of Semiconductors, Chinese Academy of Sciences, Beijing, China, in 2015. From 2015 to 2019, he was a Post-Doctoral Fellow with the I2Sense Laboratory, Department of Electrical and Computer Engineering, University of Calgary, Calgary, Canada. In 2019, he joined CenBRAIN Neurotech, School of Engineering, Westlake University, Hangzhou, China, where